ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. Balamurugan S, for providing valuable guidance, continuous support and encouragement throughout the development of this mini project.

We are thankful to the Department of Computer Science and Information Technology, JAIN (Deemed-to-be University), for giving us the opportunity to develop this project as part of the Cloud Computing subject.

We would also like to acknowledge the developers and documentation teams of React, TypeScript, Vite, Tailwind CSS, Node.js, Express.js, Socket.IO, Recharts, Jest, Vercel and Render for providing the technologies and services used in this project. We also acknowledge the authors of the original Isolation Forest paper, whose work made the threat detector possible.

Finally, we would like to thank our faculty members, friends and everyone who provided suggestions, feedback and support throughout the successful completion of this project.

ABSTRACT

PLEXO Traffic Core is a full-stack, browser-based simulator of a distributed cloud load balancer with integrated machine-learning threat detection. It models a pool of five microservice nodes behind a load balancer, generates synthetic traffic of adjustable rate and pattern, and streams every routing decision, metric and security event to a live React dashboard.

The simulator implements two routing algorithms, Round Robin and Least Connections, which can be switched at run time. Chaos-engineering controls crash and recover nodes and inject traffic spikes, so that fault tolerance and elasticity can be observed directly. Every request is converted into a six-dimensional feature vector describing its sender's behaviour and is scored by an Isolation Forest of 25 trees, implemented from scratch in TypeScript without any machine-learning library. Clients whose anomaly score reaches 0.80 are banned automatically, and the ban log can be exported as JSON. Because the model is unsupervised, no labelled attack data is needed.

The project also includes an algorithm laboratory that demonstrates Dijkstra's shortest path for datacentre routing and the 0/1 knapsack for compute scheduling, a statistical analytics module with percentile latencies, a Pearson correlation matrix and descriptive statistics, and a CART decision-tree latency predictor with SLA indication.

The server uses Node.js, Express and Socket.IO, the client uses React, Vite and TailwindCSS, and the system is verified by 29 automated tests. A deployment path using Vercel for the front end and Render for the back end is documented. The project integrates concepts from Algorithms and Data Structures, Cloud Computing and Software Data Analytics, and demonstrates load balancing, resource pooling, fault tolerance, measured service and cloud security in a single interactive system.

Keywords: Cloud Computing, Load Balancing, Round Robin, Least Connections, Isolation Forest, Anomaly Detection, Chaos Engineering, Socket.IO, React, Node.js.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 9

1.1 Introduction and Background 9

1.2 Problem Statement 9

1.3 Motivation for Selecting the Project 10

1.4 Objectives of the Project 10

1.5 Scope of the Project 11

1.6 Significance of the Project 11

1.7 Organisation of the Report 11

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEMS 12

2.1 Introduction 12

2.2 Load Balancing in Cloud Computing 12

2.3 Fault Tolerance and Chaos Engineering 12

2.4 Algorithms for Distributed Systems 13

2.5 Anomaly Detection for Network Security 13

2.6 Regression Trees for Performance Prediction 14

2.7 Review of Existing Systems 14

2.8 Comparison of Existing Approaches 14

2.9 Limitations of Existing Approaches 15

2.10 Proposed System 15

2.11 Advantages of the Proposed System 15

CHAPTER 3: REQUIREMENTS ANALYSIS 16

3.1 Introduction 16

3.2 Functional Requirements 16

3.3 Non-Functional Requirements 17

3.4 Hardware Requirements 17

3.5 Software Requirements 17

3.6 User Roles and Use Cases 18

3.7 Feasibility Analysis 18

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE 19

4.1 Introduction 19

4.2 Design Goals 19

4.3 Overall System Architecture 19

4.4 Client Architecture 20

4.5 Server Architecture 20

4.6 Request Lifecycle and Data Flow 21
4.7 Communication Interfaces 21
4.8 In-Memory Data Model 22
4.9 Module Interaction 23
4.10 Technology Choices 23
CHAPTER 5: IMPLEMENTATION 24
5.1 Introduction 24
5.2 Development Environment 24
5.3 Traffic Generator 24
5.4 Simulation Engine 25
5.5 Isolation Forest Threat Detector 26
5.6 CART Decision Tree Latency Predictor 29
5.7 ADA Algorithm Laboratory 31
5.8 Analytics Module 32
5.9 Client Implementation 33
CHAPTER 6: CLOUD COMPUTING CONCEPTS AND DEPLOYMENT 34
6.1 Introduction 34
6.2 Cloud Computing Concepts Demonstrated by PLEXO 34
6.3 Service and Deployment Models 34
6.4 Target Deployment Architecture 35
6.5 Frontend Deployment on Vercel 35
6.6 Backend Deployment on Render 35
6.7 Configuration and Environment Variables 36
6.8 Challenges in Cloud Deployment 36
6.9 Security Considerations 37
CHAPTER 7: TESTING AND VALIDATION 38
7.1 Introduction 38
7.2 Testing Strategy 38
7.3 Automated Test Suite 38
7.4 Manual Functional Test Cases 39
7.5 Threat Detection Validation 40
7.6 Performance Observations 40
7.7 Testing Conclusion 40
CHAPTER 8: RESULTS AND DISCUSSION 41
8.1 Introduction 41

8.2 Experimental Setup 41
8.3 Load Balancing View 41
8.4 Traffic Analytics View 42
8.5 Threat Detection 43
8.6 Results Obtained 44
8.7 Challenges and Solutions 44
8.8 Discussion 45
8.9 Current Limitations 45
CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS 46
9.1 Conclusion 46
9.2 Achievement of Objectives 46
9.3 Learning Outcomes 46
9.4 Future Enhancements 47
REFERENCES / BIBLIOGRAPHY 48
APPENDIX A: APPLICATION DETAILS 49
A.1 Project Structure 49
APPENDIX B: BASIC USER MANUAL 50
B.1 Installation 50
B.2 Using the Dashboard 50
B.3 Production Build 50

LIST OF FIGURES

Figure 4.1: System architecture of PLEXO Traffic Core 19
Figure 4.2: Request lifecycle through the threat detector and routing engine 21
Figure 5.1: Anomaly score against average path length for psi = 64 28
Figure 5.2: Predictor response to connections at three CPU levels (100,000 req/s) 30
Figure 5.3: Feature importances of the CART latency model 30
Figure 5.4: Worked example of Dijkstra's algorithm, shortest path DC-A to DC-E in red 31
Figure 6.1: Target cloud deployment topology of PLEXO 35
Figure 8.1: Load Balancing view with Round Robin routing across five nodes 41
Figure 8.2: Traffic Analytics view with the engine in standby 42
Figure 8.3: Traffic Analytics view with the engine active 43
Figure 8.4: Attack injection controls in the Threat Radar 43

LIST OF TABLES

Table 2.1: Comparison of existing approaches with PLEXO 14
Table 3.1: Functional requirements 16
Table 3.2: Hardware requirements 17
Table 3.3: Software requirements 17
Table 3.4: Principal use cases 18
Table 4.1: Server modules and responsibilities 20
Table 4.2: REST API endpoints 21
Table 4.3: Server to client events 22
Table 4.4: Client to server events 22
Table 4.5: Principal in-memory data structures 22
Table 4.6: Technology choices and rationale 23
Table 5.1: Development tools 24
Table 5.2: Traffic patterns 24
Table 5.3: Simulated attack types 25
Table 5.4: Isolation Forest parameters 26
Table 5.5: Feature vector 27
Table 5.6: Dijkstra trace from DC-A (distances in ms) 31
Table 5.7: Illustrative task set for the knapsack example 32
Table 5.8: Dynamic-programming table for the example (budget 8) 32
Table 5.9: Analytics components 32
Table 5.10: Application views 33
Table 6.1: Mapping of cloud concepts to PLEXO features 34
Table 6.2: Typical configuration values 36
Table 6.3: Deployment challenges and mitigations 36
Table 7.1: Testing approaches 38
Table 7.2: ThreatDetector.test.ts 38
Table 7.3: SimulationEngine.test.ts 38
Table 7.4: TrafficGenerator.test.ts 39
Table 7.5: Manual functional test cases 39
Table 9.1: Objectives and how they were achieved 46
Table 9.2: Application details 49

CHAPTER 1: INTRODUCTION

1.1 Introduction and Background

Cloud computing has changed the way software is built, deployed and operated. Instead of buying and maintaining physical servers, organisations rent computing capacity, storage and networking from cloud providers and scale it up or down as demand changes. A consequence of this model is that a single application is rarely served by a single machine. Requests from thousands of users are spread across a pool of servers, and the component that decides which server receives which request is the **load balancer**.

A load balancer is one of the most important building blocks of any cloud architecture. It improves availability, because a failed server can be removed from rotation without users noticing. It improves performance, because work is shared between several machines rather than queued on one. It also improves scalability, because capacity can be added by placing more servers behind the balancer. Production systems such as AWS Elastic Load Balancing, Google Cloud Load Balancing and NGINX all implement variations of a small set of routing algorithms, most commonly Round Robin and Least Connections.

At the same time, a load balancer sits at the edge of the system and is therefore the first component to be exposed to hostile traffic. Volumetric floods, port scanning and payload-based exploits all arrive through the same entry point as legitimate requests. Rule-based filters can block attacks that are already known, but they struggle with new behaviour. Modern platforms therefore increasingly use data analytics and machine learning to separate normal traffic from abnormal traffic automatically.

PLEXO Traffic Core is a full-stack, browser-based simulator that brings these two concerns together. It models a pool of five microservice nodes behind a load balancer, generates synthetic traffic with a controllable rate and pattern, allows faults to be injected, and passes every request through an unsupervised machine learning model that scores how unusual the sender's behaviour is. All decisions made by the system are streamed to a live dashboard so that a user can observe routing, latency, node health and threat detection as they happen.

The project was developed as a postgraduate capstone that combines three subjects of the M.Sc. programme: Algorithms and Data Structures (ADA), Cloud Computing (CC) and Software Data Analytics (SDA). This report is submitted as part of the Cloud Computing subject and therefore concentrates on the cloud-computing aspects of the work, namely load balancing,

fault tolerance, elasticity, real-time telemetry and deployment, while also describing the algorithmic and analytical components that make the simulator complete.

1.2 Problem Statement

Students and practitioners who study load balancing normally meet it in one of two forms. The first is a textbook description of algorithms such as Round Robin, which is easy to understand but cannot be observed running. The second is a commercial cloud load balancer, which can be observed but is a black box: the routing decisions, the internal metrics and the failure behaviour cannot be inspected or experimented with, and using one costs money and requires cloud infrastructure.

Existing teaching simulators also tend to treat routing and security as separate subjects. A routing simulator rarely models hostile clients, and a security demonstration rarely models how the cluster behind the balancer reacts to the load. In real systems the two are tightly coupled, since an attack changes request rate, error rate, latency and connection counts all at once.

The problem addressed by this project can therefore be stated as follows: **to design and implement an interactive, real-time simulator of a distributed cloud load balancer that exposes classical routing algorithms, fault injection and live telemetry, and that integrates an unsupervised machine-learning threat detector which identifies and blocks abnormal clients without needing labelled attack data.**

1.3 Motivation for Selecting the Project

Several considerations led the team to choose this project.

- Load balancing is a core topic of the Cloud Computing syllabus, and building a working simulator is a more rigorous way to understand it than reading about it.
- The topic naturally connects three subjects. Dijkstra's algorithm and the 0/1 knapsack problem from ADA map onto inter-datacentre routing and task scheduling, while the statistical dashboard and anomaly detection map onto SDA.
- Real-time systems raise engineering questions that coursework rarely touches: how to push live state to a browser without polling, how to keep memory bounded in a long-running process, and how to keep machine-learning inference cheap enough to run on the request path.

- Implementing the Isolation Forest algorithm from first principles, without an ML library, offered a chance to understand the algorithm at the level of the original research paper.

1.4 Objectives of the Project

The objectives of PLEXO Traffic Core are:

1. To build a real-time simulator of a five-node microservice pool that routes requests using the Round Robin and Least Connections algorithms, switchable at run time.
2. To provide chaos-engineering controls that crash and recover nodes and inject traffic spikes, so that fault tolerance can be observed directly.
3. To generate synthetic traffic with configurable rate and pattern, including a small proportion of suspicious requests from persistent attacker identities.
4. To implement an Isolation Forest anomaly detector in TypeScript, with no third-party ML dependency, and use it to ban abnormal clients automatically.
5. To demonstrate two classical algorithms from ADA, Dijkstra's shortest path and the 0/1 knapsack, in the context of datacentre routing and compute scheduling.
6. To present live statistical analytics, including percentile latencies, a Pearson correlation matrix and descriptive parameters, alongside a CART-based latency predictor.
7. To stream all state to a React dashboard over WebSockets without polling, and to keep server memory bounded during long sessions.
8. To verify the implementation with an automated test suite and to document a deployment path using Vercel for the frontend and Render for the backend.

1.5 Scope of the Project

The scope of the project covers the design, implementation and testing of the simulator and its dashboard. The following areas are included:

- A Node.js and Express server hosting the simulation engine, the traffic generator, the threat detector, a REST API and a Socket.IO event bus.
- A React and TypeScript single-page application with six views: Overview Dashboard, Load Balancing, ADA Algorithm Lab, Traffic Analytics, Architecture and Models, and About the Project.

- Two routing algorithms, node failure and recovery, traffic-pattern control and a real-time event log.
- Unsupervised threat detection with automatic banning, manual unbanning, and JSON export of the ban log and the traffic log.
- A statistical analytics module and a decision-tree latency predictor.

The following are outside the scope. The nodes are simulated objects inside one Node.js process and are not real containers or virtual machines. The traffic is synthetic and no real user traffic is handled. The system does not persist data to a database; all state is held in memory for the duration of a session.

1.6 Significance of the Project

PLEXO is significant as a learning and demonstration tool because it makes invisible infrastructure behaviour visible. A user can watch Round Robin distribute requests evenly, switch to Least Connections and observe the difference, crash a node and see traffic re-routed, and then launch an attack and see the machine-learning model identify and block the attacker. Every one of these events is explained by data shown on screen.

From an engineering viewpoint the project shows that a useful anomaly detector can be implemented in a few hundred lines of dependency-free TypeScript and run on the request path with negligible overhead. From an academic viewpoint it demonstrates how three subjects, algorithms, cloud computing and data analytics, combine in a single coherent system.

1.7 Organisation of the Report

The remainder of the report is organised as follows. Chapter 2 reviews existing systems and the theory behind the techniques used. Chapter 3 states the requirements. Chapter 4 presents the system architecture and data flow. Chapter 5 describes the implementation of each module in detail, including the algorithms and machine-learning models. Chapter 6 discusses the cloud deployment approach. Chapter 7 reports testing and validation. Chapter 8 presents results with screenshots and discusses challenges. Chapter 9 concludes the report and lists future enhancements.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEMS

2.1 Introduction

This chapter reviews the concepts that PLEXO builds on. It begins with load balancing in cloud systems, then covers fault tolerance and chaos engineering, graph and optimisation algorithms used in distributed systems, anomaly detection for network security, and regression trees for performance prediction. It closes by comparing existing approaches with the proposed system.

2.2 Load Balancing in Cloud Computing

A load balancer distributes incoming requests across a group of back-end servers. Balancers operate at different layers of the network stack. A Layer 4 balancer works on transport-level information such as IP addresses and ports and forwards connections without inspecting their content, which makes it very fast. A Layer 7 balancer inspects application-level data such as HTTP headers and URLs, which allows content-based routing and request filtering at the cost of more processing. The PLEXO interface describes itself as L4/L7 ready, reflecting that the simulated balancer exposes both connection counts and request-level attributes.

2.2.1 Round Robin

Round Robin is the simplest dynamic algorithm. The balancer keeps a pointer into the list of servers and sends each new request to the next server, wrapping round at the end. Selection takes constant time and requires no knowledge of server state. Its weakness is that it assumes all requests cost the same and all servers have equal capacity. If some requests are long-running, one server can accumulate many open connections while another sits idle.

2.2.2 Least Connections

Least Connections addresses that weakness by sending each new request to the server with the fewest active connections. The balancer must track connection counts and scan the healthy servers, so selection takes O(N) time for N servers, but the algorithm adapts automatically when requests vary in duration. It is generally preferred for heterogeneous workloads.

2.2.3 Other Algorithms

Production balancers also offer Weighted Round Robin, which allows servers of different capacity to receive proportionally different shares, IP Hash and consistent hashing, which keep

a given client on the same server, and Power of Two Choices, which samples two servers at random and picks the less loaded. These are noted as future extensions in Chapter 9.

2.3 Fault Tolerance and Chaos Engineering

Distributed systems fail in partial and unpredictable ways. Chaos engineering is the discipline of deliberately injecting failures, such as terminating servers or adding latency, in order to verify that the system degrades gracefully. The practice was popularised by Netflix's Chaos Monkey. A load balancer is expected to detect an unhealthy node, remove it from rotation, continue serving from the remaining nodes and re-admit the node when it recovers. PLEXO reproduces this behaviour with explicit Crash Node and Recover controls and a fault-injection panel.

2.4 Algorithms for Distributed Systems

2.4.1 Dijkstra's Shortest Path

Dijkstra's algorithm computes the minimum-cost path from a source vertex to all other vertices in a graph with non-negative edge weights. Using a binary min-heap as the priority queue, it runs in O((V + E) log V) time. It is the basis of link-state routing protocols such as OSPF, in which each router builds a map of the network and computes shortest paths over it. In a multi-datacentre cloud, edge weights can represent inter-datacentre latency, and the shortest path gives the lowest-latency route between two sites.

2.4.2 The 0/1 Knapsack Problem

In the 0/1 knapsack problem each item has a weight and a value, and a subset must be chosen to maximise total value without exceeding a weight limit. Each item is either taken whole or left out. Dynamic programming solves it in O(n x W) time and space by building a table in which entry (i, w) stores the best value achievable using the first i items with capacity w. In cloud scheduling the items are queued tasks, the weight is the resource cost and the value is a priority score, so the problem models admission of work under a fixed compute budget.

2.5 Anomaly Detection for Network Security

Signature-based intrusion detection compares traffic with a database of known attack patterns. It is accurate for known attacks but blind to new ones. Supervised machine-learning classifiers need labelled examples of both normal and attack traffic, and such labelled data is scarce,

expensive to produce and quickly outdated as attackers change tactics. Unsupervised anomaly detection avoids both problems by learning what normal behaviour looks like and flagging whatever differs from it.

2.5.1 Isolation Forest

Isolation Forest was proposed by Liu, Ting and Zhou in 2008. Most anomaly detectors build a profile of normal data and report points that deviate from it. Isolation Forest instead exploits the fact that anomalies are few and different, and are therefore easier to separate from the rest of the data. The algorithm builds many random binary trees. At each node a feature is picked at random and a split value is picked at random between that feature's minimum and maximum. Recursion continues until a point is alone in its leaf or a depth limit is reached. A typical anomaly is isolated after very few splits, so its average path length across the forest is short, while a typical normal point needs many splits.

Because it does not compute distances or densities, Isolation Forest has low computational cost, scales to many dimensions and needs no labels. These properties suit a request-path detector where inference must take well under a millisecond.

2.6 Regression Trees for Performance Prediction

Classification and Regression Trees (CART), introduced by Breiman and colleagues in 1984, partition the feature space with a sequence of binary splits chosen to minimise a cost criterion; for regression the criterion is mean squared error. Each leaf stores a predicted value. Trees capture threshold effects and interactions between variables that a linear model cannot represent. This is relevant to server performance because latency usually stays low until a resource nears saturation and then rises sharply.

2.7 Review of Existing Systems

2.7.1 Commercial Cloud Load Balancers

AWS Elastic Load Balancing, Azure Load Balancer and Google Cloud Load Balancing provide production-grade balancing, health checks and auto-scaling. They are managed services whose internal decisions are not visible, they incur cost, and they are not designed for experimentation.

2.7.2 Open-Source Software Balancers

NGINX, HAProxy and Envoy are widely used open-source balancers. They expose rich configuration and logs but require installation, configuration files and real back-end servers, and they offer no built-in visual dashboard or integrated machine-learning detector.

2.7.3 Academic and Teaching Simulators

Cloud simulation frameworks such as CloudSim model data-centre resource allocation and scheduling in detail, but they are batch-oriented Java toolkits that produce results after a run rather than an interactive live view. Small classroom demonstrations of load balancing usually show only the routing step.

2.8 Comparison of Existing Approaches

Table 2.1: Comparison of existing approaches with PLEXO

Feature	**Commercial LB**	**Open-source LB**	**Batch simulator**	**PLEXO**
Routing algorithms	Yes	Yes	Yes	Yes (2, switchable live)
Live visual dashboard	Provider console	Needs add-ons	No	Yes
Fault injection in UI	Limited	No	Scripted	Yes
Visible internal decisions	No	Partial (logs)	After run	Yes, real time
Integrated ML threat detection	Separate product	No	No	Yes (Isolation Forest)
Algorithm teaching (Dijkstra, Knapsack)	No	No	No	Yes
Cost / infrastructure needed	Paid cloud	Servers to configure	Java toolchain	Free, runs in browser

2.9 Limitations of Existing Approaches

- Managed balancers hide their internals, so they are poor teaching tools and cannot be freely experimented with.

- Open-source balancers need real servers and configuration, which raises the barrier to entry.
- Batch simulators do not provide an interactive real-time view of routing and failures.
- Routing and security are usually treated separately, so the effect of an attack on routing metrics is not shown.
- Supervised security models require labelled attack data that a newly deployed system does not have.

2.10 Proposed System

PLEXO Traffic Core is proposed to fill these gaps. It is an open, inspectable, browser-based simulator in which routing algorithms, node health, traffic patterns and attacks can all be controlled from a single dashboard, and in which every request is evaluated by an unsupervised Isolation Forest whose decisions, scores and feature vectors are visible and exportable.

2.11 Advantages of the Proposed System

1. Integrates routing, fault tolerance, analytics and security in one system.
2. Streams all state to the browser over WebSockets, so the dashboard is truly real time.
3. Needs no labelled attack data, because the Isolation Forest is unsupervised.
4. Implements the detector with no ML library, which keeps it lightweight and fully transparent.
5. Bounds memory use, so it can run for long sessions without degrading.
6. Costs nothing to run and can be started locally with two commands.

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Introduction

Requirements analysis identifies what the system must do and the conditions under which it must do it. This chapter lists the functional and non-functional requirements, the hardware and software needed, the user roles and a feasibility assessment.

3.2 Functional Requirements

Functional requirements describe the operations the system must perform. They are grouped below by subsystem.

Table 3.1: Functional requirements

ID	Requirement	Description
FR01	Start and stop simulation	The user can start, pause and reset the simulation engine from the dashboard.
FR02	Round Robin routing	Requests are dispatched to healthy nodes in a fixed circular order.
FR03	Least Connections routing	Requests are dispatched to the healthy node with the fewest active connections.
FR04	Switch algorithm at run time	The routing algorithm can be changed without restarting the simulation.
FR05	Node failure and recovery	A selected node can be crashed and later recovered; failed nodes receive no traffic.
FR06	Traffic generation control	The user can set request rate (1 to 250 req/s) and choose Constant, Random, Burst or Spike patterns.
FR07	Node monitoring	Each node shows health status, active connections, CPU, memory, request count and latency.
FR08	Cluster metrics	Total dispatched, completed, dropped, mean, p50, p95, p99 and minimum latency are displayed.
FR09	Real-time event log	Each routing decision is streamed to an event log with request, node and time.
FR10	Threat scoring	Every request is scored by the Isolation Forest using a six-dimensional feature vector.

ID	Requirement	Description
FR11	Automatic banning	A client whose score reaches the threshold (0.80) is banned and its later requests are intercepted.
FR12	Manual threat controls	The user can unban one client, clear all bans, or inject a simulated attack of a chosen type.
FR13	JSON export	The ban log and the traffic log can be downloaded as JSON files.
FR14	Dijkstra demonstration	The user can choose source and destination datacentres and view the shortest path and its cost.
FR15	Knapsack demonstration	The user can set a compute budget and view the optimal task subset and the DP table.
FR16	Statistical analytics	The system shows throughput and latency charts, a Pearson correlation matrix and descriptive statistics.
FR17	Latency prediction	The user can set RPS, connections and CPU with sliders and see predicted latency and SLA status.

3.3 Non-Functional Requirements

3.3.1 Performance

Metrics must be pushed to the client every 500 ms, and threat scoring must add negligible delay to request processing.

3.3.2 Reliability and Memory Safety

The server must remain stable over long sessions. Per-client history buffers and the banned-client list must therefore be bounded, so that memory usage reaches a plateau rather than growing without limit.

3.3.3 Usability, Maintainability and Security

The interface must be clear and responsive, with a persistent sidebar and colour coding that makes node health and threat state recognisable at a glance. The code base must be written in TypeScript with shared types, separated into client, server and simulation modules, covered by automated tests, and portable to any machine with Node.js 18 or higher. State-changing operations such as banning must pass through a defined REST and WebSocket interface, and production deployments must use HTTPS with a restricted CORS origin.

3.4 Hardware Requirements

Table 3.2: Hardware requirements

Component	Minimum requirement
Processor	Any modern dual-core processor
RAM	4 GB (8 GB recommended for development)
Network	Local loopback is sufficient; internet needed only to install packages or deploy

3.5 Software Requirements

Table 3.3: Software requirements

Software	Version	Purpose
Node.js	18 or higher (20+ recommended)	Server runtime and build tooling
React	18.2	Frontend UI library
TypeScript	5.4	Typed language for client and server
Vite	5.1	Frontend dev server and bundler
TailwindCSS	3.4	Utility-first styling
Recharts	2.12	Charts
Express	4.18	REST API
Socket.IO	4.7	Real-time WebSocket communication
Jest and ts-jest	29.7	Automated testing
Web browser	Chrome, Edge, Firefox or Brave	Access to the dashboard

3.6 User Roles and Use Cases

PLEXO has a single interactive role, the **operator** (the person running the simulation, for example a student or evaluator). There is no login, because the simulator is a local teaching and demonstration tool.

Table 3.4: Principal use cases

Use case	Actor	Outcome
Start and observe routing	Operator	Requests flow to nodes and metrics update live.
Compare Round Robin and Least Connections	Operator	Distribution of connections changes visibly after switching.
Crash a node	Operator	Node turns red, traffic redistributes to the remaining nodes.
Inject an attack	Operator	Detector scores abnormal requests and bans the attacker.
Review banned clients	Operator	Table shows score, counts, reason and time; client can be unbanned.
Export logs	Operator	JSON files are downloaded for offline analysis.
Run Dijkstra or Knapsack	Operator	Algorithm result and steps are visualised.

3.7 Feasibility Analysis

Technical feasibility. All technologies used are mature, well documented and free. TypeScript is used on both client and server, and the Isolation Forest algorithm is simple enough to implement from the published paper with low enough inference cost for a Node.js process. **Economic feasibility.** The project uses only open-source software and free hosting tiers; development and demonstration need only a personal computer. **Operational feasibility.** The application is operated through a web browser, starts with two commands, and has labelled controls with live feedback.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Introduction

This chapter describes how PLEXO is structured. It presents the overall architecture, the responsibilities of each module, the data flow of a single request, the communication interfaces (REST and WebSocket), and the in-memory data structures used in place of a database.

4.2 Design Goals

- **Separation of concerns.** Routing, traffic generation, threat detection and transport are in separate modules.
- **Push, not poll.** The server pushes state to the browser through WebSockets, so the interface updates immediately and no polling loops are needed.
- **Single source of truth.** All simulation state lives in the server. The client holds only a reactive copy in one hook.
- **Bounded resources.** Every buffer that grows with traffic has an upper limit.
- **Observability.** Every decision, from node choice to ban score, is exposed to the user.

4.3 Overall System Architecture

PLEXO is a two-tier client-server application. The client is a React single-page application served by Vite on port 5173 during development. The server is a Node.js process on port 3001 that hosts an Express HTTP server and a Socket.IO server on the same port. The server contains four main modules: the socket handler, the simulation engine, the traffic generator and the threat detector.

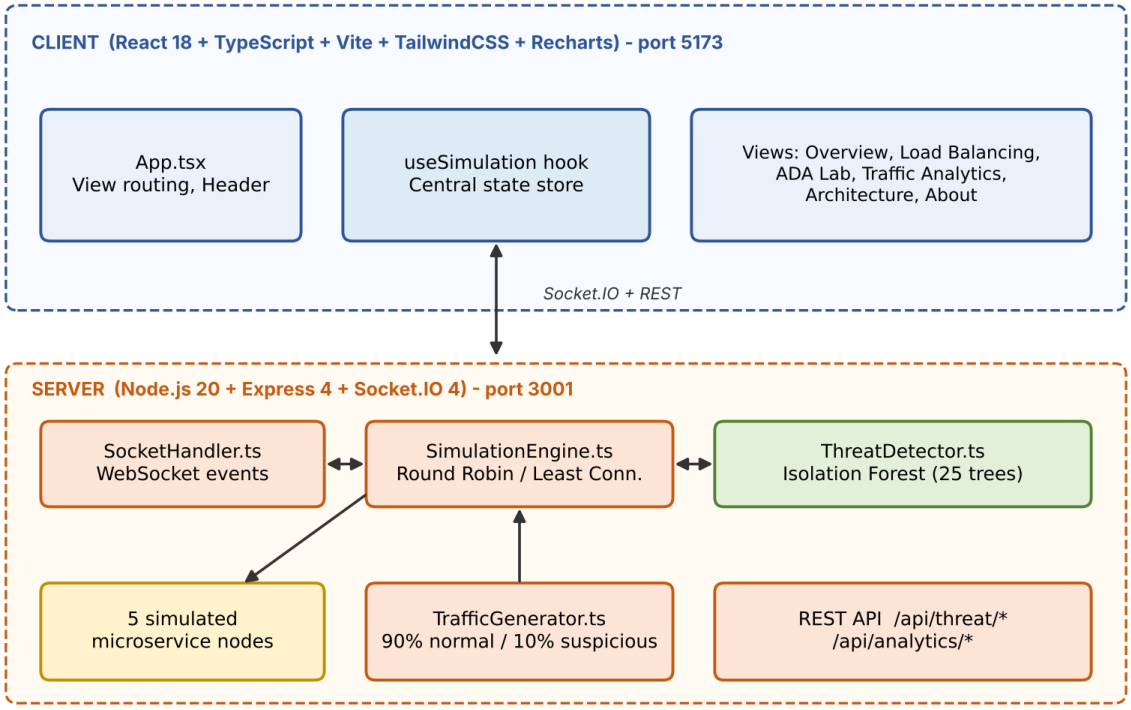


Figure 4.1: System architecture of PLEXO Traffic Core

The user interacts with the React views. User actions such as Start, Spike or Crash Node are sent to the server as Socket.IO events. The server updates the simulation, and metrics, events and threat notifications flow back to the client as Socket.IO events. Operations that are naturally request and response, such as fetching the banned list or clearing bans, use the REST API.

4.4 Client Architecture

The client is organised around three ideas.

- **App.tsx** is the root component. It renders the header, the sidebar navigation and the currently selected view.
- **useSimulation** is a custom React hook that owns the Socket.IO connection and the REST client. It stores metrics, node states, the event log, the time-series buffers and the list of banned clients, and exposes them to the views. Because every view reads from this single hook, switching views never loses data.
- **View components** present the data: OverviewDashboardView, CloudLoadBalancingView, ADALabView, TrafficAnalyticsView, CloudArchitectureView and AboutProjectView.

Styling uses TailwindCSS utility classes, charts use Recharts, and icons come from Lucide React. The Traffic Analytics view uses a local state variable to switch between a Statistics tab and an ML Models and Security tab, so that a large amount of content is organised without overwhelming a single page.

4.5 Server Architecture

Table 4.1: Server modules and responsibilities

Module	File	Responsibility
Entry point	index.ts	Creates the Express app and HTTP server, attaches Socket.IO, registers routes.
Socket handler	websocket/SocketHandler.ts	Receives control events from the client and emits metrics, events and threat updates.
REST routes	routes/api.ts	Implements /api/threat/* and /api/analytics/* endpoints.
Simulation engine	simulation/SimulationEngine.ts	Maintains nodes, applies the routing algorithm, tracks health, computes cluster metrics, orchestrates threat checks.
Traffic generator	simulation/TrafficGenerator.ts	Synthesises normal and suspicious requests, including the persistent attacker pool.
Threat detector	simulation/ThreatDetector.ts	Isolation Forest training, scoring, banning and memory-bounded client history.
Shared types	types/index.ts	Interfaces and enums shared across the server.

4.6 Request Lifecycle and Data Flow

Every simulated request follows the same path, shown in Figure 4.2. The traffic generator creates a request carrying a client identifier and payload attributes. The engine records it in that client's history buffer and derives a six-dimensional feature vector. The Isolation Forest scores the vector. If the score is at or above 0.80, the client is banned, its feature vector is stored with the ban record, and a threat:client_banned event is emitted; subsequent requests from the client are intercepted and counted as blocked. Otherwise the engine chooses a healthy node with the active routing algorithm, updates that node's connection count and latency, updates cluster metrics, and emits events to the client.

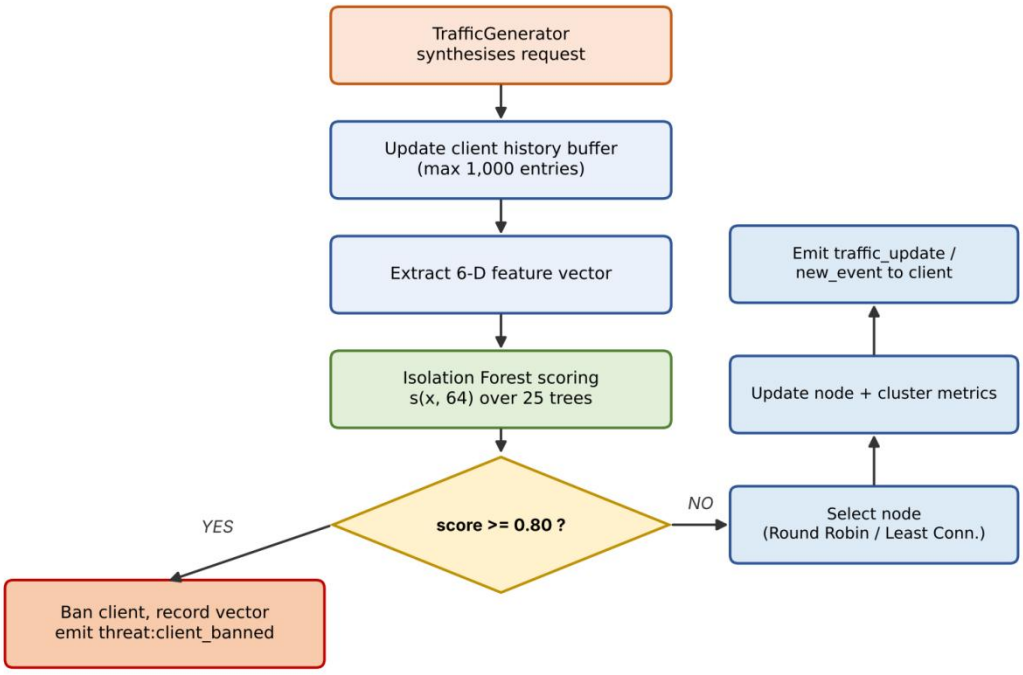


Figure 4.2: Request lifecycle through the threat detector and routing engine

4.7 Communication Interfaces

4.7.1 REST API

The Express server exposes the following endpoints on port 3001.

Table 4.2: REST API endpoints

Method	Endpoint	Description
GET	/api/threat/banned	Returns the list of currently banned clients.
POST	/api/threat/ban	Manually bans a client by identifier.
DELETE	/api/threat/ban/:clientId	Unbans one client.
DELETE	/api/threat/clear	Clears all bans and returns the count cleared.
POST	/api/threat/simulate	Triggers a simulated attack of type VOLUMETRIC_FLOOD, ENTROPY_EXPLOIT or SCAN_CRAWLER.
GET	/api/analytics/report	Returns the traffic analytics report (metrics, series, nodes).
GET	/api/simulation/status	Returns running state, algorithm and uptime.

4.7.2 Socket.IO Events

Real-time communication uses named Socket.IO events. Table 4.3 lists the events sent by the server and Table 4.4 those sent by the client.

Table 4.3: Server to client events

Event	Payload	Description
traffic_update	SimulationMetrics	Metrics snapshot emitted every 500 ms.
new_event	EventLogEntry	One entry per routing decision.
threat:evaluated	ThreatEvaluationResult	Emitted after each Isolation Forest scoring.
threat:request_banned	clientId, requestId, score	A request from a banned client was intercepted.
threat:client_banned	BannedClientRecord	A client crossed the 0.80 threshold.
threat:client_unbanned	clientId	A client was unbanned.
threat:banned_list	BannedClientRecord[]	Full list after any ban-state change.

Table 4.4: Client to server events

Event	Payload	Description
start_simulation	algorithm	Starts the engine with ROUND_ROBIN or LEAST_CONNECTIONS.
stop_simulation	none	Stops the engine.
switch_algorithm	algorithm	Changes the routing algorithm at run time.
inject_spike	none	Triggers a ten-second traffic spike.
fail_node / recover_node	none	Crashes or recovers a node.
threat:unban	clientId	Unbans a client.
threat:clear_all	none	Clears all bans.

4.8 In-Memory Data Model

PLEXO does not use a database. The simulation is a live process whose state is meaningful only while it runs, and an in-memory model gives the lowest latency. The principal structures are listed below.

Table 4.5: Principal in-memory data structures

Structure	Type	Contents and bound
nodes	Array of 5 node objects	Health, active connections, CPU, memory, request count, latency.
clientHistories	Map of clientId to array	Recent requests per client; at most 1,000 entries per client (FIFO).
bannedClients	Map of clientId to record	Score, counts, reason, time and feature vector; at most 500 entries.
forest	Array of 25 isolation trees	Built once at start-up from 250 synthetic vectors; read-only afterwards.
ATTACKER_POOL	Array of 8 identifiers	Persistent attacker identities reused in 80% of suspicious traffic.
time series buffers (client)	Arrays in the hook	Throughput and latency points for the charts.

4.9 Module Interaction

The engine owns the clock. On each tick it asks the traffic generator for the requests due in that interval, passes each to the threat detector and, for those that pass, to the routing algorithm. The threat detector never calls the engine directly; it returns a result and the engine acts on it, which keeps the detector a pure, easily testable unit. The socket handler subscribes to engine callbacks and translates them into Socket.IO emissions, so the engine itself has no knowledge of the network transport.

4.10 Technology Choices

Table 4.6: Technology choices and rationale

Choice	Rationale
TypeScript everywhere	One language for client and server; shared interfaces catch mismatches at compile time.
Socket.IO	Reliable full-duplex channel with automatic reconnection; removes the need for polling.

Choice	Rationale
Vite	Very fast hot-module replacement during development and small production bundles.
TailwindCSS	Consistent design system with minimal custom CSS.
Recharts	Declarative, composable charts that integrate cleanly with React state.
Isolation Forest (custom)	Unsupervised, cheap inference, no labelled data, no library dependency.
In-memory state	Lowest latency and simplest deployment for a simulation.

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

This chapter describes how each part of PLEXO was implemented: the traffic generator, the routing engine, the Isolation Forest threat detector, the decision-tree latency predictor, the algorithm laboratory, the analytics module and the client interface. Code fragments are given as simplified pseudocode that follows the structure of the real implementation.

5.2 Development Environment

Table 5.1: Development tools

Tool	Use
Visual Studio Code	Code editing for client and server
Node.js 20 and npm	Runtime and package management
Vite dev server	Serves the client on port 5173 with hot reload
ts-node-dev	Runs the TypeScript server with automatic restart on port 3001
Jest with ts-jest	Unit and integration tests for the server
Git and GitHub	Version control
Chrome / Brave DevTools	Inspecting WebSocket frames and performance

A root-level package.json uses the concurrently utility so that npm run dev starts the server and the client together.

5.3 Traffic Generator

The traffic generator produces the synthetic workload. Each generated request carries a client identifier and the attributes from which the six features are derived: request rate, burst behaviour, payload entropy, payload size, error ratio and consecutive rejections.

5.3.1 Traffic Patterns

The user selects a request rate from 1 to 250 requests per second using a slider or preset buttons (1, 5, 10, 25, 50, 100, 250) and one of four patterns.

Table 5.2: Traffic patterns

Pattern	Behaviour
Constant	A steady rate equal to the slider value.
Random	The rate fluctuates randomly around the slider value.
Burst	Short periods of much higher rate separated by quieter periods.
Spike	A temporary multiple of the base rate (the load-balancing view and the spike control raise the rate for ten seconds).

5.3.2 Normal and Suspicious Requests

About 90% of requests are normal. Their attributes are drawn from ranges typical of benign clients: low request rates (about 0.1 to 5 per second), moderate payload sizes (128 to 4,096 bytes), low payload entropy (below 3.0), a small error ratio and at most a few consecutive rejections. About 10% of requests are suspicious, with attributes drawn from attack-like ranges.

5.3.3 The Persistent Attacker Pool

A key design decision was to make suspicious traffic come from persistent identities. In the first version every suspicious request carried a freshly generated random identifier. Because the detector builds its features from each client's history, a client seen for the first time has an empty history and a near-zero feature vector, so nothing was ever scored as anomalous and the blocked counter stayed at one. The fix was ATTACKER_POOL, a set of eight identifiers that are reused for 80% of suspicious requests. With persistent identities, evidence accumulates: request rate and consecutive rejections rise over time until the score crosses the threshold and the client is banned. This models real attackers, who return repeatedly.

5.3.4 Attack Types

Three attack profiles can be launched on demand from the Threat Radar buttons (Figure 8.4) or through the REST API.

Table 5.3: Simulated attack types

Attack	API value	Dominant features
Entropy exploit	ENTROPY_EXPLOIT	High payload entropy and large payloads (randomised or obfuscated content).
Burst flood	VOLUMETRIC_FLOOD	Very high request rate and burst velocity.

Attack	API value	Dominant features
Port scanner	SCAN_CRAWLER	High consecutive rejections and error ratio from probing invalid routes.

5.4 Simulation Engine

5.4.1 Node Model

The server pool consists of five nodes, node-1 to node-5. Each node object stores its health status (Healthy, Degraded or Offline), the number of active connections, CPU and memory utilisation, the total requests handled and a rolling average latency. The pool is shown in the dashboard as the Elastic Microservice Server Pool, described as a Layer 4 and 7 worker pool that scales and handles simulated workloads.

5.4.2 Round Robin

```
pointer = 0
function selectRoundRobin(nodes):
    for i in 0 .. nodes.length - 1:
        candidate = nodes[(pointer + i) mod nodes.length]
        if candidate.status != OFFLINE:
            pointer = (pointer + i + 1) mod nodes.length
            return candidate
    return NONE          # all nodes down
```

Selection is O(1) in the normal case and O(N) in the worst case when several consecutive nodes are offline. The skipping of failed nodes is what allows traffic to flow around a crash.

5.4.3 Least Connections

```
function selectLeastConnections(nodes):
    best = NONE
    for node in nodes:
        if node.status == OFFLINE: continue
        if best == NONE or node.activeConnections < best.activeConnections:
            best = node
    return best
```

Selection is O(N). With five nodes this is negligible, and the algorithm adapts when some nodes hold long-running connections.

5.4.4 Health, Chaos Engineering and Spikes

Crash Node sets a chosen node offline immediately; both algorithms then exclude it. Recovery returns the node to service. The Chaos Fault Injection panel provides a node selector and fault controls. The spike control temporarily raises the request rate so that the effect of a surge on connections and latency can be observed.

5.4.5 Cluster Metrics

Every 500 ms the engine computes and emits a metrics snapshot. It includes the total number dispatched, completed and dropped requests, throughput, error rate, mean latency and the median, 95th and 99th percentile latencies, the minimum latency, and the count of healthy, degraded and offline nodes. Percentiles are computed over a window of recent latency samples by sorting and indexing at the requested rank.

5.5 Isolation Forest Threat Detector

5.5.1 Why an Unsupervised Model

A supervised classifier was rejected for three reasons. First, a newly deployed balancer has no labelled attack history. Second, attacks change, and a classifier trained on known flood signatures does not detect new behaviour; Isolation Forest assumes only that anomalies are few and different. Third, inference cost: scoring requires traversing 25 trees of depth at most 6, about 150 comparisons, which is far cheaper than a large ensemble or neural network.

5.5.2 Parameters

Table 5.4: Isolation Forest parameters

Parameter	Value	Rationale
Number of trees (t)	25	Stable scores at low cost.
Sub-sample size (psi)	64	Smaller than the 256 suggested in the original paper, chosen to reduce latency.
Maximum depth	6 (= log2 of 64)	Standard depth limit for the sub-sample size.
Ban threshold	0.80	Chosen by experiment between 0.60 and 0.95 (Section 5.5.7).
Training corpus	250 synthetic benign vectors	Generated at server start-up.

5.5.3 Feature Vector

Each request is represented by a six-dimensional vector derived from the sender's recent history.

Table 5.5: Feature vector

No.	Feature	Meaning	Typical normal range
f1	reqRatePerSec	Requests from this client in the last one-second window	0.1 to 5
f2	burstVelocity	Requests in the last 100 ms scaled to a per-second figure	0 to 10
f3	payloadEntropy	Shannon entropy of the payload bytes	0.5 to 2.5
f4	payloadSizeBytes	Size of the payload	128 to 4,096
f5	errorRateRatio	Fraction of this client's requests that returned errors	0 to 0.15
f6	consecutiveRejections	Current run of consecutive 4xx or 5xx responses	0 to 3

Different attacks stand out on different features. A volumetric flood has high f1 and f2, an entropy exploit has high f3 and f4, and a port scanner has high f5 and f6. Because the forest chooses features at random, any of these can isolate the offender quickly.

5.5.4 Tree Construction and Path Length

```
function buildTree(data, depth, maxDepth):
    if depth >= maxDepth or data.size <= 1:
        return Leaf(size = data.size)
    q = random feature index in 0..5
    lo, hi = min(data[q]), max(data[q])
    if lo == hi: return Leaf(size = data.size)
    p = uniform random value in (lo, hi)
    left  = rows with value < p
    right = rows with value >= p
    return Node(q, p, buildTree(left, depth+1, maxDepth),
                      buildTree(right, depth+1, maxDepth))


function pathLength(x, node, depth):
```

```
if node is Leaf:
    return depth + c(node.size)
if x[node.q] < node.p: return pathLength(x, node.left,  depth+1)
else:                  return pathLength(x, node.right, depth+1)
```

When a point reaches a leaf that still holds several training points, the path length is corrected by c(size), the expected additional depth that would be needed to separate them. This is the standard adjustment from the original paper.

5.5.5 Anomaly Score

For a point x, the anomaly score is s(x, psi) = 2 raised to the power of minus E[h(x)] divided by c(psi). Here E[h(x)] is the mean path length of x over the 25 trees and c(psi) is the average path length of an unsuccessful search in a binary search tree of psi points:

```
c(n)  = 2 * H(n - 1) - 2 * (n - 1) / n
H(i)  = ln(i) + 0.5772156649          (Euler-Mascheroni approximation)
s(x)  = 2 ^ ( - E[h(x)] / c(64) )
```

For psi = 64, c(64) is approximately 7.47. The score has a clear interpretation: values near 1 mean the point was isolated very quickly and is strongly anomalous; values near 0.5 mean the point's path length equals the average; values well below 0.5 mean the point sits deep inside the normal data. Figure 5.1 plots the score against the mean path length. The ban threshold of 0.80 corresponds to a mean path length of only about 2.4 splits, which means a client is banned only if the forest isolates it in far fewer splits than the 7.47 average.

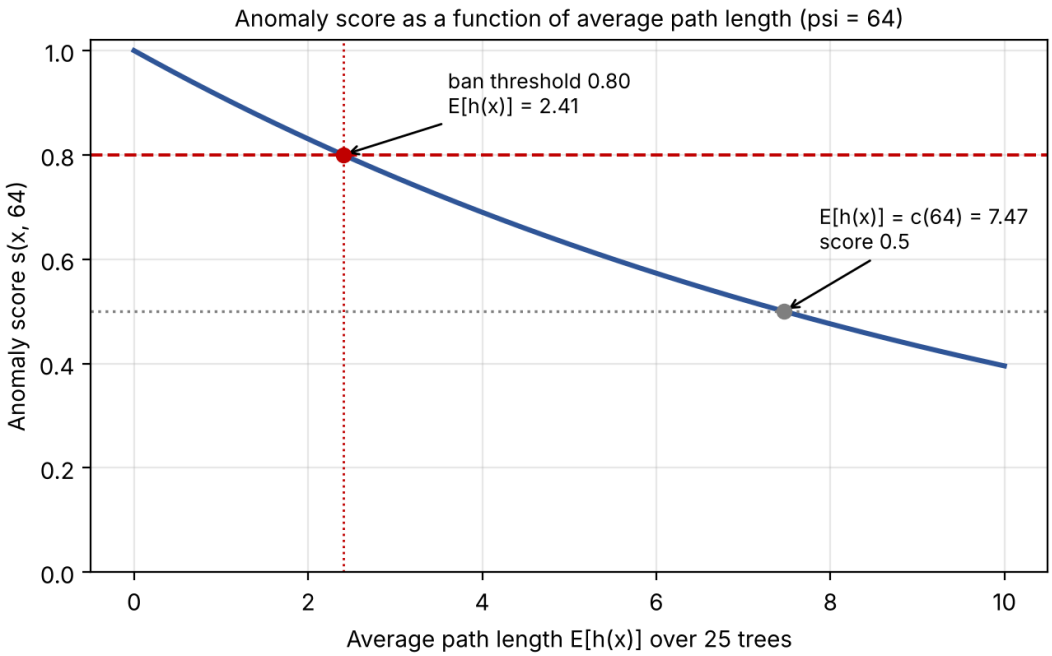


Figure 5.1: Anomaly score against average path length for psi = 64

5.5.6 Training and Scoring

At server start-up the detector generates 250 synthetic benign vectors, with each feature drawn from the normal ranges of Table 5.5, and builds the 25 trees, each from a random sub-sample of 64 vectors. The forest is then frozen; no learning happens during the simulation. At run time the pipeline for each request is: update client history, extract the vector, compute path lengths in all 25 trees, average them, compute the score, compare with 0.80, and either ban the client or allow the request through.

5.5.7 Threshold Selection

The threshold was selected experimentally by running the simulation across thresholds from 0.60 to 0.95 and counting correct bans of attacker-pool members and incorrect bans of benign clients. According to the team's development runs, 0.60 banned benign clients with temporarily bursty behaviour, and 0.95 left moderately anomalous attackers unbanned. At 0.80 no benign client was banned during extended runs, and attacker-pool members were banned within the first few minutes of simulation.

5.5.8 Memory Management

Two bounds keep the server stable. MAX_CLIENT_HISTORY limits each client's history to 1,000 entries, with the oldest entry discarded first. MAX_BANNED_CLIENTS limits the ban list

to 500 entries, evicting the oldest ban when full. As a result the memory footprint levels off shortly after start-up instead of growing for the life of the process.

5.5.9 Performance on the Request Path

Node.js runs JavaScript on a single thread, so a slow evaluation would delay WebSocket heartbeats and freeze the dashboard. With psi = 64 and t = 25, a scoring operation visits at most 25 x 6 = 150 tree nodes. In the team's benchmarking on the development machine this took consistently under 0.08 ms, so the evaluation is performed synchronously within the request-processing tick, which also keeps error handling simple.

5.6 CART Decision Tree Latency Predictor

The second model is a regression tree that predicts average response latency from three infrastructure features: ingress requests per second, active connections and CPU utilisation. It is exposed as an interactive widget in the ML Models tab of Traffic Analytics, with three sliders (RPS 0 to 250,000, connections 0 to 150,000, CPU 0 to 100%).

5.6.1 Model Description

The model follows the CART method of Breiman and colleagues, with mean squared error as the split criterion, depth 8 and cost-complexity pruning (alpha = 0.01). The decision logic extracted into the front end reproduces the dominant branches of the tree:

```
latency  = 4.2                              # base intercept
latency += (connections / 80000) * 42.0         # connections branch
latency += (rps         / 250000) * 28.0        # throughput branch
latency += (cpu         / 100)    * 18.0        # CPU branch
if connections > 50000 and cpu > 80:
   latency += 35.0                           # congestion interaction
```

The final conditional captures a non-linear interaction: when many connections coincide with high CPU, the system enters congestion and latency rises disproportionately. Figure 5.2 evaluates the predictor at 100,000 requests per second for three CPU levels. At 40% and 70% CPU the response grows gradually with connections, whereas at 90% CPU the jump of 35 ms appears once connections pass 50,000, taking the prediction above the 50 ms SLA limit.

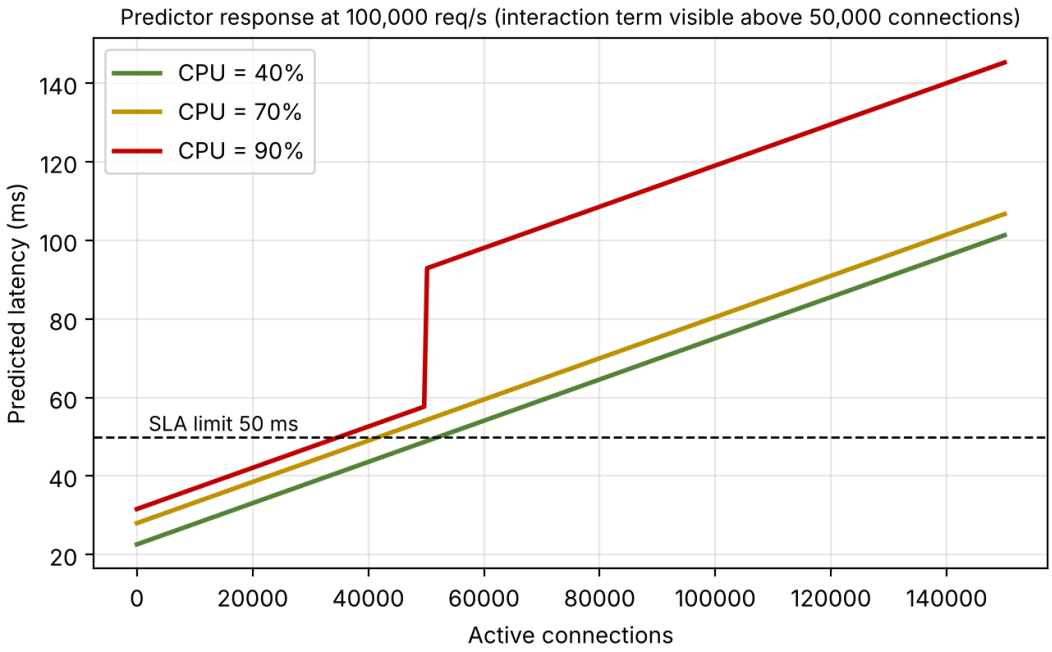


Figure 5.2: Predictor response to connections at three CPU levels (100,000 req/s)

5.6.2 Performance and Feature Importance

The performance figures reported by the project for the model are R-squared 0.942, MAE 1.84 ms and RMSE 2.42 ms, with depth 8. The feature importances are shown in Figure 5.3: active connections 42%, ingress RPS 28%, CPU utilisation 18% and payload size 12%. Connection queueing delays requests directly, which explains why connections dominate.

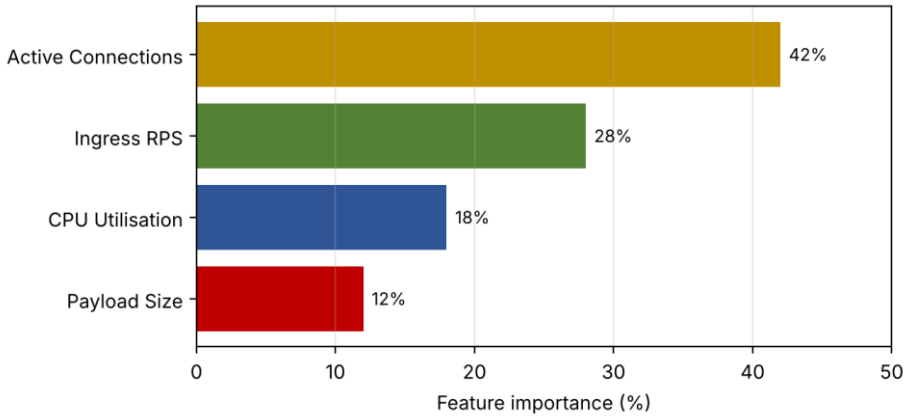


Figure 5.3: Feature importances of the CART latency model

5.6.3 Why a Tree Rather Than Linear Regression

Linear regression assumes that each feature contributes independently and proportionally. Latency does not behave that way: below roughly 50,000 connections the cluster copes, and above that, combined with high CPU, latency rises steeply. A tree represents such thresholds

as explicit splits. The project reports an R-squared of about 0.71 for linear regression on the same data, compared with 0.942 for the tree.

The predictor also displays an SLA badge: green when the predicted latency is below 50 ms and red when it is 50 ms or more.

5.7 ADA Algorithm Laboratory

5.7.1 Dijkstra's Shortest Path

The first sub-view finds the minimum-latency route between two of five datacentres. Edge weights are network latency in milliseconds. The user picks a source and a destination, and the algorithm animates each relaxation step. The implementation uses a min-heap priority queue and runs in O((V + E) log V) time and O(V + E) space.

To illustrate the algorithm, Figure 5.4 shows a worked example on a five-datacentre graph (the edge weights here are illustrative and are not taken from the application). Table 5.6 gives the state after each step; a star marks a finalised distance.

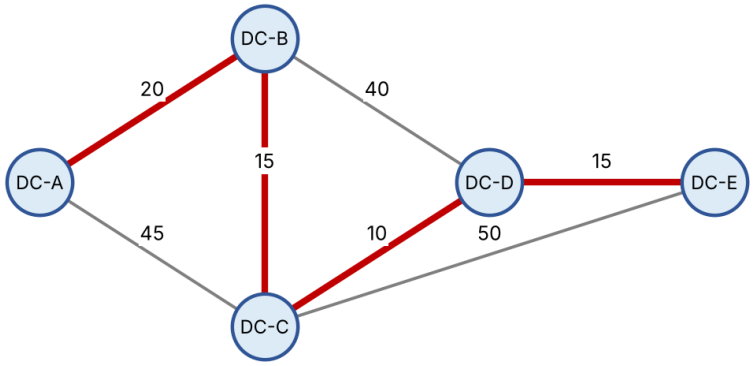


Figure 5.4: Worked example of Dijkstra's algorithm, shortest path DC-A to DC-E in red

Table 5.6: Dijkstra trace from DC-A (distances in ms)

Step	Visited	DC-A	DC-B	DC-C	DC-D	DC-E
1	DC-A	0*	20	45	inf	inf
2	DC-B	0*	20*	35	60	inf
3	DC-C	0*	20*	35*	45	85
4	DC-D	0*	20*	35*	45*	60

Step	Visited	DC-A	DC-B	DC-C	DC-D	DC-E
5	DC-E	0*	20*	35*	45*	60*

The shortest path from DC-A to DC-E is DC-A > DC-B > DC-C > DC-D > DC-E with a total cost of 60 ms. Any route that ends with the DC-C to DC-E link costs at least 85 ms, so the algorithm correctly prefers the longer chain of cheaper links. This is exactly the principle used by OSPF in real networks.

5.7.2 0/1 Knapsack Task Scheduler

The second sub-view selects, from a set of cloud tasks such as video encoding, a data ETL pipeline and an ML training job, the subset with the highest total priority that fits within a compute budget set by a slider. The dynamic-programming table is displayed so that the optimal-substructure property can be traced. The recurrence is:

```
T[i][w] = T[i-1][w]                          if weight[i] > w
T[i][w] = max( T[i-1][w],  T[i-1][w - weight[i]] + value[i] )   otherwise
```

Time and space are O(n x W). As an illustration, consider a budget of 8 units and the four tasks of Table 5.7 (values are illustrative).

Table 5.7: Illustrative task set for the knapsack example

Task	Compute cost	Priority value
Data ETL pipeline	2	3
Video encoding	3	4
ML training job	5	8
Nightly backup	4	5

Table 5.8: Dynamic-programming table for the example (budget 8)

Items	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7	w=8
none	0	0	0	0	0	0	0	0	0
1..1	0	0	3	3	3	3	3	3	3
1..2	0	0	3	4	4	7	7	7	7
1..3	0	0	3	4	4	8	8	11	12

Items	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7	w=8
1..4	0	0	3	4	5	8	8	11	12

The optimum is 12, obtained by selecting Video encoding and ML training job. Tracing back from the last cell, whenever a value differs from the row above, the item in that row was taken.

5.8 Analytics Module

The Traffic Analytics view derives its statistics from the time-series buffers in the client hook.

Table 5.9: Analytics components

Component	Description
Telemetry cards	Requests analysed, mean latency, minimum latency, maximum peak latency, p99 latency, standard deviation (jitter) and error rate.
Dual-scope chart	Ingress throughput (k req/s) against latency (ms) over a rolling window, with annotated anomaly flags such as a DDoS burst and a failover event.
Pearson correlation matrix	Pairwise linear correlation of server resource metrics (RPS, connections, latency, error rate, payload size).
Descriptive parametrics	Mean, median, standard deviation, interquartile range, skewness and kurtosis of the latency series.

The Pearson correlation coefficient between two series X and Y is r = cov(X, Y) / (sigma_X x sigma_Y), which lies between -1 and +1. Values near +1 indicate that the two metrics rise together; near -1 that one rises as the other falls. The matrix lets the user see which resource metrics move with latency. Skewness measures asymmetry of the latency distribution and kurtosis measures tail heaviness; a high kurtosis suggests occasional extreme latency spikes.

5.9 Client Implementation

5.9.1 The useSimulation Hook

The hook opens the Socket.IO connection once, registers handlers for each server event and writes incoming data into React state. It also exposes action functions (start, stop, switch algorithm, crash node, inject attack, unban, clear) that emit events or call REST endpoints. Because all views share this hook's state, the application is fully reactive with no polling.

5.9.2 Views

Table 5.10: Application views

View	Content
Overview Dashboard	Bento-grid summary of requests, latency, nodes and throughput, algorithm status and navigation cards.
Load Balancing	Algorithm selector, traffic generator, chaos fault injection, five-node pool, live dispatch flow and latency chart.
ADA Algorithm Lab	Dijkstra and Knapsack visualisations.
Traffic Analytics	Statistics tab and ML Models and Security tab.
Architecture and Models	System diagrams, complexity table, model cards and deployment architecture.
About Project	Academic context, contributions, objectives, methodology and references.

5.9.3 Threat Radar and Export

The ML Models and Security tab lists banned clients with their threat score, requests sent, requests blocked, ban reason and ban time, each with an Unban button. It also provides Clear All Bans and two export buttons. Exports are created entirely in the browser with the Blob API: the traffic log is saved as plexo_traffic_log_<timestamp>.json and the ban log as plexo_ban_log_<timestamp>.json. The ban log stores each banned client's feature vector, which allows later analysis of which behaviour triggered the ban.

```
{
 "exportedAt": "...",
 "totalBanned": 3,
 "bannedClients": [{
  "clientId": "attacker-entropy_exploit-5b73e399",
  "threatScore": 0.94,
  "requestsSent": 847,
  "requestsBlocked": 391,
  "banReason": "Anomaly score 0.94 exceeded threshold 0.80",
  "featureVector": { "reqRatePerSec": 42.3, "payloadEntropy": 7.9, ... }
 }]
}
```

The JSON above is an example of the export format.

CHAPTER 6: CLOUD COMPUTING CONCEPTS AND DEPLOYMENT

6.1 Introduction

Because this report is submitted for the Cloud Computing subject, this chapter relates PLEXO to the core ideas of cloud computing. It first shows how the simulator demonstrates those ideas, and then describes the deployment approach for hosting the application on cloud platforms. The NIST definition of cloud computing lists on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service as its essential characteristics, and these are used below as a frame of reference.

6.2 Cloud Computing Concepts Demonstrated by PLEXO

Table 6.1: Mapping of cloud concepts to PLEXO features

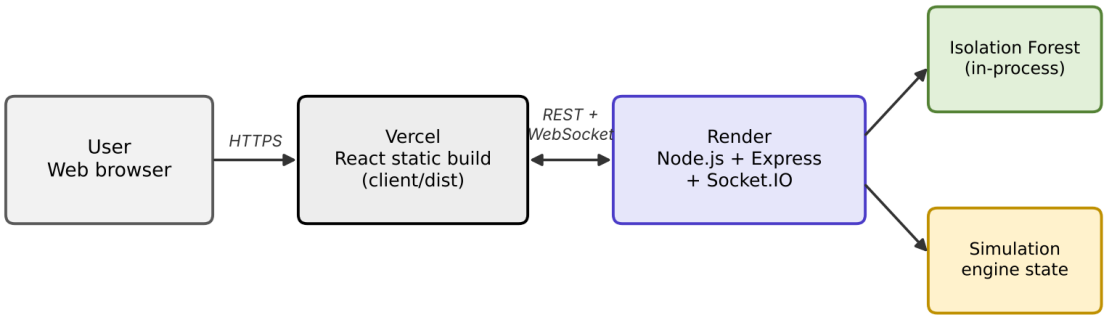
Cloud concept	How PLEXO demonstrates it
Resource pooling	Five microservice nodes form a shared server pool; requests are drawn from many clients and served by whichever node the balancer selects.
Load balancing	Round Robin and Least Connections are implemented and can be switched live, with visible effect on per-node connections and latency.
High availability	A crashed node is excluded from routing and the remaining nodes continue to serve; the cluster health figure is displayed in the sidebar.
Fault tolerance and chaos engineering	Crash Node, recovery and the Chaos Fault Injection panel let the operator create failures and watch the system degrade gracefully.
Elasticity	The traffic generator scales from 1 to 250 req/s and spike injection raises demand suddenly, showing how the pool reacts to variable load.
Measured service	Throughput, error rate, p50, p95, p99 latency and per-node utilisation are continuously measured and displayed.
Microservice architecture	The pool is presented as independent worker services behind a single entry point, and the client and server are independently deployable.
Cloud security	The Isolation Forest detects and blocks abnormal clients at the edge, before they consume back-end capacity.

6.3 Service and Deployment Models

PLEXO is itself a web application, and the platforms chosen for hosting it illustrate the cloud service models. Vercel, which hosts static front-end bundles and serves them from a content-delivery network, and Render, which runs a Node.js web service from a Git repository, are both examples of **Platform as a Service (PaaS)**: the provider manages servers, operating systems and runtime patching, and the developer supplies only code and configuration. Both are public-cloud services. The simulated load balancer, by contrast, models the **Infrastructure as a Service (IaaS)** layer, where virtual servers are rented and traffic distribution must be managed.

6.4 Target Deployment Architecture

The architecture page of the application documents a deployment in which the front end and back end are hosted separately. The React build is deployed to Vercel, and the Node.js, Express and Socket.IO server is deployed to Render. The Isolation Forest and the simulation engine run inside the Render process.



Target deployment topology. Demonstration runs in this report were executed locally (localhost:5173 and localhost:3001).

Figure 6.1: Target cloud deployment topology of PLEXO

The demonstrations and screenshots in this report were produced by running the application locally, with the client on localhost:5173 and the server on localhost:3001. The hosted deployment described in this chapter is the intended production configuration; the steps below document how it is carried out.

6.5 Frontend Deployment on Vercel

1. Push the repository to GitHub.

2. Create a Vercel project and import the repository, selecting the client directory as the project root.
3. Set the framework preset to Vite, the build command to `npm run build` and the output directory to `dist`.
4. Add an environment variable that holds the public URL of the backend, so that the client connects to the hosted server rather than localhost.
5. Deploy. Vercel builds the bundle and serves it from its global CDN over HTTPS.
6. Each later push to the main branch triggers an automatic redeployment.

6.6 Backend Deployment on Render

1. Create a Render Web Service from the same GitHub repository, with the server directory as the root.
2. Set the build command to `npm install && npm run build` and the start command to run the compiled server from `dist`.
3. Configure environment variables, such as the port and the allowed front-end origin used for CORS.
4. Deploy and verify from the logs that the HTTP and Socket.IO servers started.
5. Update the Vercel environment variable with the Render service URL and redeploy the front end.

Socket.IO shares the HTTP port with Express, so no second port or service is required. Render web services support WebSocket connections, which is necessary for the real-time channel.

6.7 Configuration and Environment Variables

Table 6.2: Typical configuration values

Value	Where set	Purpose
Backend URL	Vercel (client)	Tells the client which server to contact for REST and Socket.IO.
Port	Render (server)	Port on which the server listens; Render supplies it.
Allowed origin	Render (server)	Front-end origin permitted by CORS and Socket.IO.
Node environment	Render (server)	Selects production behaviour.

The simulator does not use a database or third-party API keys, so there are no credentials to protect. If persistent logging or an external service were added, their secrets would be stored only as server-side environment variables and never in client code or the repository.

6.8 Challenges in Cloud Deployment

Table 6.3: Deployment challenges and mitigations

Challenge	Mitigation
Cross-origin requests between Vercel and Render are blocked by default.	Configure CORS and the Socket.IO origin to allow only the deployed front-end URL.
Free Render instances spin down after inactivity, delaying the first request by up to a minute.	Accept for demonstrations, or use a paid instance or a keep-alive ping.
Simulation state is held in memory and is lost on restart or redeploy.	Acceptable for a simulator; persistent logging is listed as a future enhancement.
The client must connect to the correct backend URL in each environment.	Use an environment variable rather than a hard-coded address.
A single server process cannot be scaled horizontally without sharing state.	Future work: externalise state and use a Socket.IO adapter.
Mixed content errors if HTTP is used from an HTTPS page.	Serve the backend over HTTPS and use secure WebSocket connections.

6.9 Security Considerations

- Serve all public traffic over HTTPS and WSS.
- Restrict CORS to the known front-end origin.
- Validate the inputs of REST endpoints such as /api/threat/ban.
- Apply rate limiting and authentication to the control endpoints if the deployment is made public, since anyone could otherwise ban clients or start attacks.
- Keep dependencies up to date and avoid exposing stack traces in error responses.
- Bound memory-consuming structures, as PLEXO already does, to avoid denial of service through resource exhaustion.

CHAPTER 7: TESTING AND VALIDATION

7.1 Introduction

Testing verifies that the simulator behaves according to its requirements. PLEXO was tested at two levels: an automated unit and integration test suite for the server, and manual functional testing of the dashboard.

7.2 Testing Strategy

Table 7.1: Testing approaches

Approach	What it verifies
Unit testing	Individual functions such as the c(n) constant, tree depth bounds and score range.
Component testing	The threat detector, simulation engine and traffic generator as isolated modules.
Integration testing	Engine and detector together, including ban events passed through callbacks.
Functional testing	End-to-end behaviour from the browser: controls, views, exports.
Statistical validation	Behaviour of the generator and detector over many samples, for example the proportion of suspicious requests.

7.3 Automated Test Suite

The server contains 29 automated tests in three suites, written with Jest and ts-jest and run with `npm test` from the server directory. Tables 7.2 to 7.4 list the tests of each suite.

Table 7.2: ThreatDetector.test.ts

No.	Test	Result
1	Initialises with 25 trees	Pass
2	Returns a score between 0 and 1 for any input	Pass
3	Scores a synthetic attack vector higher than a benign vector	Pass
4	Auto-bans a client when the score exceeds 0.80	Pass
5	Correctly unbans a client	Pass

No.	Test	Result
6	Does not exceed MAX_BANNED_CLIENTS	Pass
7	Rotates client history when MAX_CLIENT_HISTORY is exceeded	Pass
8	c(n) normalisation constant is positive for n > 1	Pass
9	Path length is bounded by the maximum tree depth	Pass
10	Does not ban the same client twice	Pass

Table 7.3: SimulationEngine.test.ts

No.	Test	Result
1	Initialises with 5 healthy nodes	Pass
2	Routes requests using Round Robin	Pass
3	Routes to the least connected node under LEAST_CONNECTIONS	Pass
4	Excludes failed nodes from routing	Pass
5	Tracks the total request count	Pass
6	Updates active connections correctly	Pass
7	Handles simulateMaliciousAttack without throwing	Pass
8	Emits the banned event through the callback	Pass

Table 7.4: TrafficGenerator.test.ts

No.	Test	Result
1	Generates approximately 10% suspicious requests	Pass
2	Reuses ATTACKER_POOL identifiers at about 80%	Pass
3	Suspicious requests have higher reqRatePerSec	Pass
4	Suspicious requests have higher payloadEntropy	Pass
5	Normal requests have payloadEntropy below 3.0	Pass
6	Does not generate duplicate request IDs	Pass
7	ATTACKER_POOL contains exactly 8 entries	Pass
8	Feature vectors are 6-dimensional	Pass
9	Benign client IDs do not appear in ATTACKER_POOL	Pass

No.	Test	Result
10	Payload size is within documented bounds	Pass
11	Error-rate ratio is bounded in [0, 1]	Pass

The summary reported by the test runner is:

```
Test Suites: 3 passed, 3 total
Tests:       29 passed, 29 total
```

7.4 Manual Functional Test Cases

The following test cases were applied to the running dashboard.

Table 7.5: Manual functional test cases

ID	Test case	Expected result	Status
TC01	Open http://localhost:5173	Dashboard loads with sidebar and header	Pass
TC02	Click Start	Engine becomes active; requests dispatched count increases	Pass
TC03	Select Round Robin	Requests spread evenly across nodes	Pass
TC04	Select Least Connections	New requests favour the least loaded node	Pass
TC05	Change request rate slider and presets	Throughput follows the selected rate	Pass
TC06	Choose Burst and Spike patterns	Rate and latency rise visibly	Pass
TC07	Crash a node	Node shows offline; traffic redistributes	Pass
TC08	Recover the node	Node returns to healthy and receives traffic	Pass
TC09	Inject Entropy Exploit	Attacker scored and banned; appears in banned table	Pass
TC10	Inject Burst Flood	Attacker banned; blocked count increases	Pass
TC11	Inject Port Scanner	Attacker banned; blocked count increases	Pass
TC12	Click Unban on a client	Client removed from banned list	Pass
TC13	Click Clear All Bans	Banned list becomes empty	Pass
TC14	Export ban log and traffic log	Two JSON files are downloaded	Pass

ID	Test case	Expected result	Status
TC15	Run Dijkstra between two datacentres	Shortest path highlighted with total cost	Pass
TC16	Run knapsack with different budgets	Optimal task subset highlighted	Pass
TC17	Move predictor sliders	Predicted latency and SLA badge update immediately	Pass
TC18	Switch between Statistics and ML Models tabs	Each tab shows its own content	Pass

7.5 Threat Detection Validation

The detector was validated in three ways. First, unit tests confirm that a synthetic attack vector scores higher than a benign vector and that a score above 0.80 triggers a ban. Second, the generator tests confirm that the suspicious-traffic proportion and the attacker-pool reuse rate match their design values, so the detector faces the intended mixture. Third, extended runs of the whole system were observed while varying the ban threshold, which led to the choice of 0.80 as described in Section 5.5.7.

7.6 Performance Observations

During development the team measured the cost of an Isolation Forest evaluation and found it to be consistently under 0.08 ms on the development machine, small compared with the Node.js event-loop tick. The dashboard remained responsive while receiving metrics every 500 ms at rates up to the 250 req/s limit of the generator. Formal load testing of large concurrent loads was not performed and is listed as a limitation.

7.7 Testing Conclusion

All 29 automated tests pass, and the manual functional cases behave as specified. The tests give good confidence in the correctness of the routing logic, the detector and the generator. They do not replace large-scale performance testing or testing against real network traffic, which are outside the scope of this simulation project.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Introduction

This chapter presents the outcome of the project through screenshots of the running application and discusses what they show. It then describes the main engineering challenges met during development and the limitations that remain.

8.2 Experimental Setup

The application was run locally on a Windows laptop, with the server on port 3001, the client on port 5173 and the dashboard in a Chromium-based browser. The traffic generator used its default mixture of 90% normal and 10% suspicious requests.

8.3 Load Balancing View

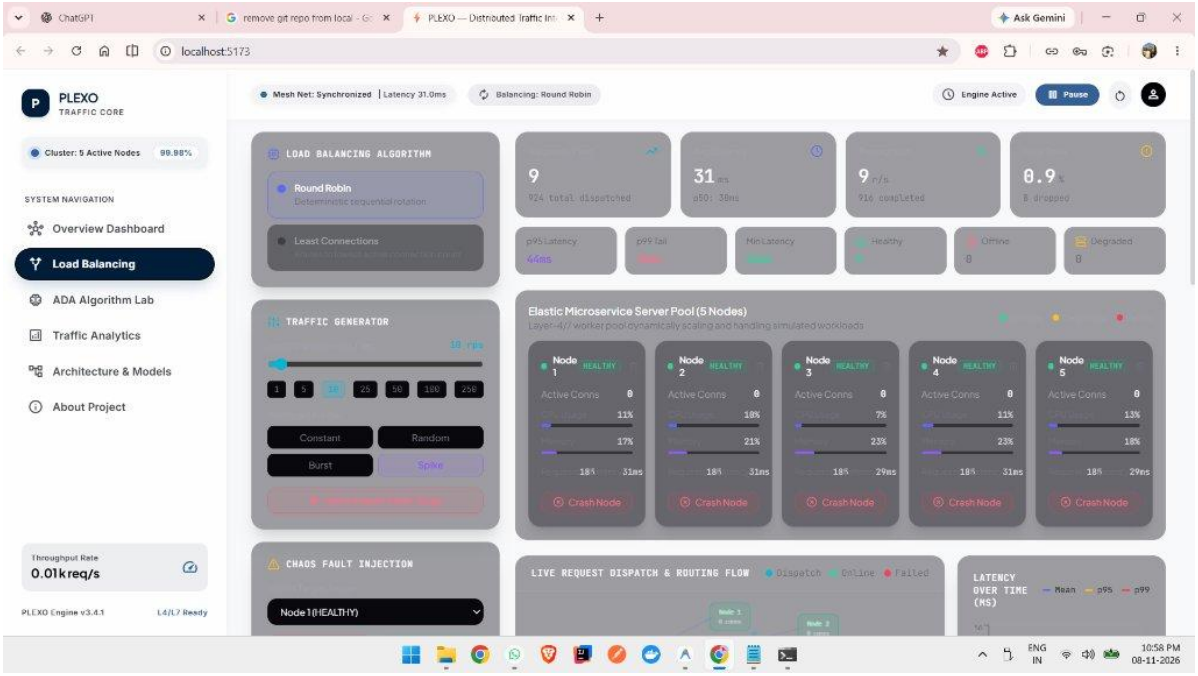


Figure 8.1: Load Balancing view with Round Robin routing across five nodes

Figure 8.1 shows the Load Balancing view during a run. The header reports that the mesh is synchronised, the balancing mode is Round Robin and the engine is active, and the sidebar shows a cluster of five active nodes at 99.98%. The Traffic Generator panel is set to 10 requests per second. The summary cards report 924 requests dispatched in total with 916 completed, a mean latency of 31 ms with a median (p50) of 30 ms, a throughput of 9 requests per second

and an error rate of 0.9% corresponding to 8 dropped requests. The 95th-percentile latency is 44 ms.

The Elastic Microservice Server Pool shows all five nodes as healthy. Each node card reports its active connections, CPU and memory utilisation, request count and latency. The nodes show similar request counts and latencies of 29 to 31 ms, which is the expected outcome of Round Robin: with identical nodes and a steady rate, the work is shared almost equally. Each card has a Crash Node button, and the Chaos Fault Injection panel below the traffic generator allows a node to be selected for fault injection.

8.4 Traffic Analytics View

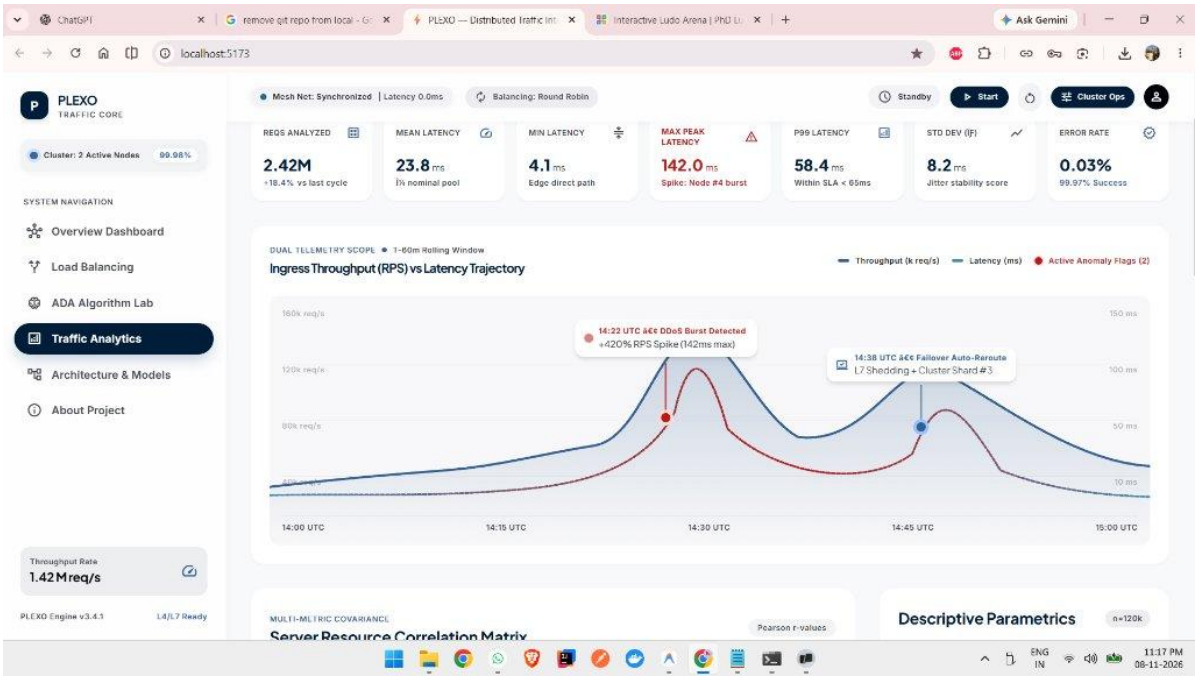


Figure 8.2: Traffic Analytics view with the engine in standby

Figure 8.2 shows the Traffic Analytics view with the engine in standby. The telemetry cards show 2.42 million requests analysed, a mean latency of 23.8 ms, a minimum of 4.1 ms, a maximum peak latency of 142.0 ms, a 99th-percentile latency of 58.4 ms (within the 65 ms SLA shown on the card), a standard deviation of 8.2 ms and an error rate of 0.03%, equivalent to 99.97% success.

The central chart, titled Ingress Throughput against Latency Trajectory, plots throughput (in thousands of requests per second) and latency (in milliseconds) over a rolling window. Two events are annotated on it: a DDoS burst at 14:22 UTC, described as a 420% requests-per-second spike with latency reaching 142 ms, and a failover auto-reroute at 14:38 UTC with

layer-7 shedding. The chart makes the relationship between a load surge and latency easy to see, as the latency curve rises and falls with the throughput curve, and the Active Anomaly Flags legend shows two flagged events. Below the chart are the Server Resource Correlation Matrix and the Descriptive Parametrics panel.

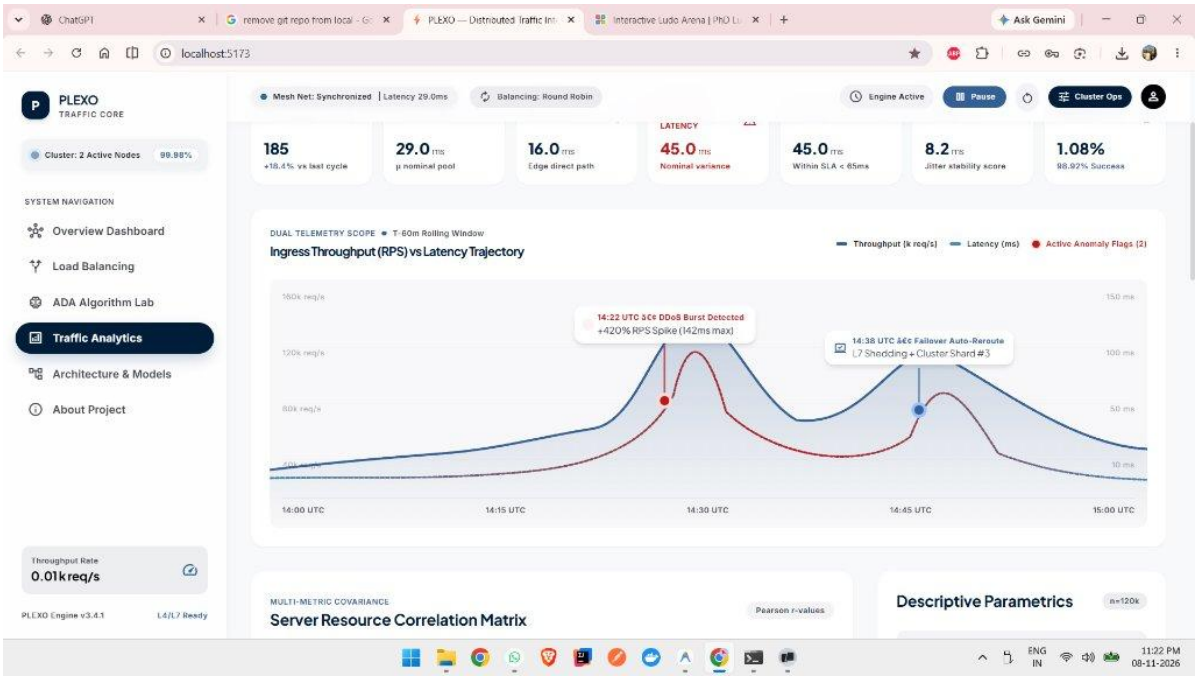


Figure 8.3: Traffic Analytics view with the engine active

Figure 8.3 shows the same view while the engine is active and the simulation is running. The cards now reflect the live run: 185 requests analysed, a mean latency of 29.0 ms, a minimum of 16.0 ms, a peak and 99th-percentile latency of 45.0 ms (within the 65 ms SLA), a jitter score of 8.2 ms and an error rate of 1.08% (98.92% success). The header latency of 29.0 ms matches the mean on the card. The Pause button replaces Start, confirming that the engine is running.

8.5 Threat Detection

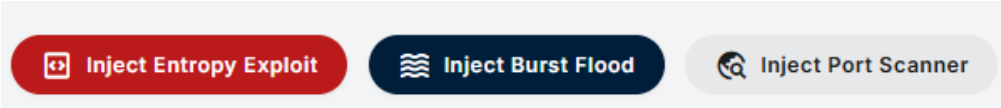


Figure 8.4: Attack injection controls in the Threat Radar

Figure 8.4 shows the three attack-injection controls, Inject Entropy Exploit, Inject Burst Flood and Inject Port Scanner. Each starts a different attack profile, corresponding to the ENTROPY_EXPLOIT, VOLUMETRIC_FLOOD and SCAN_CRAWLER types of the REST API. When an attacker's anomaly score reaches 0.80, the client appears in the banned-clients

table with its score, requests sent, requests blocked, ban reason and time, and the header shows a red ML Intercept pill with the number of banned clients.

In the project's development runs, members of the attacker pool were banned within the first few minutes of simulation, and no benign client was banned at the threshold of 0.80. An exported ban record of the kind produced by the system lists a client with a threat score of 0.94, 847 requests sent and 391 blocked, together with the feature vector that triggered the ban, in which the payload entropy of 7.9 and the error ratio of 0.63 lie far outside the normal ranges of Table 5.5.

8.6 Results Obtained

1. A working five-node load-balancer simulator with Round Robin and Least Connections routing, switchable at run time.
2. Chaos-engineering controls that crash and recover nodes and inject traffic spikes.
3. A traffic generator with adjustable rate and patterns and a persistent attacker pool.
4. An Isolation Forest detector implemented from scratch in TypeScript, with 25 trees, a 0.80 threshold and automatic banning.
5. A CART latency predictor with SLA indication and reported R-squared of 0.942.
6. Interactive Dijkstra and knapsack demonstrations.
7. A real-time analytics dashboard with percentile latencies, a correlation matrix and descriptive statistics.
8. JSON export of ban and traffic logs.
9. A test suite of 29 passing automated tests.

8.7 Challenges and Solutions

8.7.1 Building Isolation Forest From Scratch

No ML library was used, so tree construction, random split selection, path-length calculation and the normalisation constant c(psi) had to be implemented directly. The team worked from the original 2008 paper, implementing a recursive tree with internal and leaf nodes, a recursive builder that samples a random feature and split, a path-length function that adds the c(size) correction at leaves, and the harmonic-number approximation for c(n). The result is about 300 lines of TypeScript with no runtime dependencies.

8.7.2 JSX Imbalance in a Large Component

TrafficAnalyticsView grew beyond 900 lines of JSX as tabs, the correlation matrix, the statistics panel, the predictor and the threat table were added. The opening and closing div tags became unbalanced, and the compiler reported only a vague error far from the real fault. The team wrote a small Node.js script that read the file line by line, tracked a running nesting depth for div tags and reported the line where the depth went wrong. It pinpointed one surplus closing tag within a second.

8.7.3 Real-Time ML on a Single-Threaded Server

If scoring blocked the event loop for several milliseconds per request, WebSocket heartbeats would stall and the dashboard would freeze. Keeping psi at 64 and the number of trees at 25 limits an evaluation to about 150 node visits, which measured under 0.08 ms. The evaluation therefore runs synchronously in the request tick, avoiding an extra asynchronous hop and simplifying error handling.

8.7.4 Realistic Repeat-Attacker Behaviour

Fresh random identifiers for every suspicious request meant that every attacker looked new, with empty history, so nothing was ever flagged. The persistent ATTACKER_POOL of eight identities, reused 80% of the time, let evidence accumulate and made the detection meaningful.

8.7.5 Memory Management in Long Sessions

The map of client histories would grow without limit as new client identifiers were generated. Two bounds were added: 1,000 entries per client with first-in first-out discard, and 500 banned clients with oldest-first eviction. Memory now levels off within the first few minutes.

8.7.6 Organising a Dense Analytics Page

Showing statistics and machine-learning content on one page was overwhelming, while two separate pages would break the logical grouping of analytics and security. A tab switcher driven by local state (statistics or models) renders two independent subtrees inside one component, receiving the same props, which keeps the interface clean.

8.8 Discussion

The results show that a small, dependency-free implementation can combine routing, fault injection, analytics and anomaly detection in one real-time system. Several observations deserve comment.

- **Unsupervised detection works only with behavioural history.** The attacker-pool experience shows that an anomaly detector depends on features built over time. A stateless detector looking at single requests would have missed the attackers.
- **Threshold choice trades false positives against false negatives.** The 0.80 threshold corresponds to a mean path length of about 2.4 splits against an average of about 7.5, which is a demanding criterion and explains the absence of benign bans.
- **Simulation fidelity is limited but sufficient.** The nodes do not process real work, but they reproduce the effects that matter for the study: shared load, connection counts, latency and failure.

8.9 Current Limitations

1. The nodes are simulated objects in one process, not independent containers or machines.
2. Traffic is synthetic. Real traffic contains far more varied behaviour, and the detector's performance on it is unknown.
3. The Isolation Forest is trained once on synthetic benign data and is not updated during a session.
4. The CART predictor's decision logic is embedded in the front end as extracted branches, and its accuracy figures were obtained from the project's own simulation data rather than from an external benchmark.
5. State is held in memory and is lost when the server restarts; there is no persistent store.
6. Control endpoints have no authentication, so the deployment is suitable for demonstration rather than public exposure.
7. Large-scale concurrency and load testing have not been performed.
8. A minor character-encoding artefact appears in the chart annotation labels, where a separator character renders as garbled text. It does not affect function.

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

PLEXO Traffic Core is a full-stack, real-time simulator of a distributed cloud load balancer with integrated machine-learning threat detection. It distributes synthetic traffic over a pool of five nodes using Round Robin or Least Connections, lets the operator crash and recover nodes and inject spikes and attacks, and presents the effects on a live dashboard. Every request is scored by an Isolation Forest, implemented from scratch in TypeScript, which bans clients whose behaviour is abnormal without needing any labelled attack data.

The project brings together three subjects. From Algorithms and Data Structures it uses Dijkstra's shortest path, the 0/1 knapsack and a min-heap. From Cloud Computing it uses load balancing, resource pooling, fault tolerance, chaos engineering, measured service and PaaS deployment. From Software Data Analytics it uses descriptive statistics, correlation analysis, anomaly detection and regression-tree prediction.

Several engineering lessons emerged. Behavioural history is essential for detecting repeat attackers. Bounded buffers are necessary for any long-running service. Computation on a single-threaded request path must be kept small. And a clean separation between the engine, the detector and the transport layer makes a real-time system easier to test and extend. The 29 automated tests and the manual functional testing give good confidence in the correctness of the implementation.

9.2 Achievement of Objectives

Table 9.1: Objectives and how they were achieved

Objective	Status	Where addressed
Round Robin and Least Connections, switchable live	Achieved	Section 5.4, Figure 8.1
Chaos controls: crash, recover, spike	Achieved	Section 5.4.4
Configurable traffic with attacker pool	Achieved	Section 5.3
Isolation Forest in TypeScript with auto-ban	Achieved	Section 5.5
Dijkstra and knapsack demonstrations	Achieved	Section 5.7

Objective	Status	Where addressed
Live analytics and CART predictor	Achieved	Sections 5.6 and 5.8, Figures 8.2 and 8.3
WebSocket streaming and bounded memory	Achieved	Sections 4.7 and 5.5.8
Automated tests and a documented deployment path	Achieved	Chapters 6 and 7

9.3 Learning Outcomes

1. Understanding how load balancers distribute traffic and how Round Robin and Least Connections differ in behaviour and cost.
2. Applying fault-tolerance and chaos-engineering ideas to a distributed system.
3. Implementing a machine-learning algorithm from a research paper without library support.
4. Applying Dijkstra's algorithm and dynamic programming to realistic cloud problems.
5. Building a real-time full-stack application with React, TypeScript, Express and Socket.IO.
6. Using descriptive statistics, correlation and regression for system analytics.
7. Writing unit tests for stochastic and stateful code.
8. Understanding PaaS deployment of a front end and a back end on separate platforms.
9. Debugging large component trees and managing memory in long-running processes.

9.4 Future Enhancements

1. **Real containerised nodes.** Replace simulated nodes with Docker containers orchestrated by Kubernetes, so that routing acts on real services.
2. **More routing algorithms.** Weighted Round Robin, IP Hash with consistent hashing and Power of Two Choices.
3. **Auto-scaling.** Add or remove nodes automatically based on load, to demonstrate elasticity directly.
4. **Trained CART model in the browser.** Export a properly trained tree to ONNX and run it with onnxruntime-web instead of extracted branches.

5. **Adaptive detection.** Periodically retrain the Isolation Forest on recent benign traffic, and compare it with other detectors such as One-Class SVM or autoencoders.
6. **Persistent logging.** Store events and bans in SQLite or PostgreSQL for multi-session replay and analysis.
7. **Authentication and rate limiting** on the control endpoints for public deployment.
8. **Horizontal scaling** of the server using a shared store and a Socket.IO adapter.
9. **Load testing** with real tools to measure behaviour at much higher concurrency.
10. **CI/CD pipeline** that runs the test suite and deploys to Vercel and Render automatically.

REFERENCES / BIBLIOGRAPHY

1. Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413-422.
2. Breiman, L., Friedman, J., Olshen, R. and Stone, C. (1984). Classification and Regression Trees. Wadsworth, Belmont, CA.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. Introduction to Algorithms. MIT Press (chapters on shortest paths and dynamic programming).
4. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269-271.
5. Moy, J. (1998). OSPF Version 2. IETF RFC 2328.
6. Mell, P. and Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145.
7. Basiri, A. et al. (2016). Chaos Engineering. IEEE Software, 33(3), 35-41.
8. Calheiros, R. N. et al. (2011). CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Software: Practice and Experience, 41(1), 23-50.
9. React Documentation. Available at: https://react.dev/
10. TypeScript Documentation. Available at: https://www.typescriptlang.org/docs/
11. Node.js Documentation. Available at: https://nodejs.org/docs/
12. Express.js Documentation. Available at: https://expressjs.com/
13. Socket.IO Documentation. Available at: https://socket.io/docs/
14. Vite Documentation. Available at: https://vitejs.dev/guide/
15. Tailwind CSS Documentation. Available at: https://tailwindcss.com/docs
16. Recharts Documentation. Available at: https://recharts.org/
17. Jest Documentation. Available at: https://jestjs.io/docs/getting-started
18. Vercel Documentation. Available at: https://vercel.com/docs
19. Render Documentation. Available at: https://render.com/docs
20. MDN Web Docs - WebSockets API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

APPENDIX A: APPLICATION DETAILS

Table 9.2: Application details

Particulars	Details
Project name	PLEXO Traffic Core
Project type	Distributed load balancer simulator with ML-based threat detection
Frontend	React 18.2, TypeScript 5.4, Vite 5.1, TailwindCSS 3.4, Recharts 2.12, Lucide React
Backend	Node.js 20+, Express 4.18, Socket.IO 4.7, uuid 9.0
ML models	Isolation Forest (custom, 25 trees, psi = 64); CART regression tree (depth 8)
Algorithms	Round Robin, Least Connections, Dijkstra (min-heap), 0/1 Knapsack (DP)
Testing	Jest and ts-jest 29.7; 29 tests in 3 suites
Client address (local)	http://localhost:5173
Server address (local)	http://localhost:3001
Target hosting	Vercel (frontend), Render (backend)
Subjects integrated	ADA, Cloud Computing, Software Data Analytics

A.1 Project Structure

```
root/  package.json (concurrently scripts)
 client/src/  App.tsx, types/, hooks/useSimulation.ts,
              services/{api,socket}.ts, components/Header.tsx,
              components/views/{OverviewDashboard, CloudLoadBalancing,
              ADALab, TrafficAnalytics, CloudArchitecture, AboutProject}View.tsx
 server/src/  index.ts, types/, routes/api.ts, websocket/SocketHandler.ts,
              simulation/{SimulationEngine, ThreatDetector,
              TrafficGenerator}.ts, simulation/__tests__/ (3 test files)
```

APPENDIX B: BASIC USER MANUAL

B.1 Installation

1. Install Node.js (version 18 or higher) and npm.
2. Clone the repository and open a terminal in its root folder.
3. Run npm install in the root, then in the server folder and in the client folder.
4. Run npm run dev from the root to start the server and the client together, or run npm run dev separately in the server and client folders.
5. Open http://localhost:5173 in a web browser.

B.2 Using the Dashboard

1. **Start the simulation.** Click Start in the header. The Start button changes to Pause and the engine status shows Engine Active.
2. **Choose an algorithm.** Open Load Balancing and select Round Robin or Least Connections.
3. **Set traffic.** Use the slider or a preset (1 to 250 req/s) and choose Constant, Random, Burst or Spike.
4. **Inject faults.** Press Crash Node on a node card, or use the Chaos Fault Injection panel, and observe the redistribution of traffic. Recover the node when finished.
5. **View analytics.** Open Traffic Analytics. The Statistics tab shows telemetry, charts, the correlation matrix and descriptive statistics.
6. **Review threats.** Open the ML Models and Security tab. Use the Inject buttons to launch an attack, and watch the banned-clients table. Use Unban or Clear All Bans to lift bans.
7. **Export data.** Use Export Banned Log (JSON) or Export Normal Log (JSON) to download the files.
8. **Run the algorithm lab.** Open ADA Algorithm Lab and choose Dijkstra or Knapsack.
9. **Run the tests.** In the server folder run npm test.

B.3 Production Build

```
cd server && npm run build && cd ..
cd client && npm run build && cd ..
# client output: client/dist     server output: server/dist
```